COMP2300 Set 6 Practice Exam

Topics: Branch-and-Link, Calling Conventions, Stack, ABI, Recursion

Time: 120 minutes **Total:** 100 marks

Instructions

- Answer in English.
- Part A is concept-focused multiple choice.
- Part B contains computational and trace-based questions.
- Part C contains adapted past-exam style questions with sources.
- Assume 32-bit ARM unless stated otherwise.

Part A: Multiple Choice (30 marks, 3 marks each)

Choose exactly one option for each question.

A1. The main extra job performed by `BL` compared with a regular branch is to:

(A) Zero the stack pointer
(B) Save the return address in `LR`
(C) Save all caller registers automatically
(D) Write the target into `R0`

A2. In the course convention, the first four function arguments are typically passed in:

(A) `R4-R7`
(B) `R8-R11`
(C) `R0-R3`
(D) `R12-R15`

A3. Which register conventionally holds a function return value?

(A) `R0`
(B) `R4`
(C) `R12`
(D) `LR`

A4. Which statement about the stack in the lecture material is correct?

(A) It is a FIFO queue

(B) It is used only for recursion

(C) It is used to save registers, pass extra arguments, and hold local data

(D) It is part of the ISA but not the ABI

A5. Which pair best describes the division of responsibility in a calling convention?

(A) Hardware-save and software-save

(B) Caller-save and callee-save

(C) Decode-save and execute-save

(D) Stack-save and heap-save

A6. Which register is the stack pointer?

(A) `R11`

(B) `R12`

(C) `R13`

(D) `R14`

A7. Which statement about `ABI` is correct?

(A) It is identical to the ISA

(B) It includes procedure-call rules such as calling convention

(C) It is only about cache organization

(D) It defines transistor-level behavior

A8. A recursive function is always:

(A) A leaf function

(B) A function with no local variables

(C) A non-leaf function that calls itself

(D) A function that does not use the stack

A9. Which of the following is preserved across a correct function call under a calling convention?

(A) Every temporary register automatically

(B) Every argument register automatically

(C) Any callee-saved register that the callee uses, after it is restored

(D) The entire data memory

A10. A stack frame is:

(A) The set of all registers in the CPU

(B) The portion of stack space allocated for one active function call

(C) The instruction encoding of `BL`

(D) The branch target address

Part B: Computational Problems (50 marks)

B1. Branch target address (8 marks)
An instruction at address `0x2040` is a branch with `imm24 = 0x000005`.

(a) Compute `imm24 « 2`.

(b) Compute the sign-extended offset.

(c) Compute the branch target address using the ARM rule from the lecture.

B2. Register roles in a call (8 marks)
Consider the function call:

```
MOV R0, #10
MOV R1, #20
BL foo
```

The caller still needs the old values of `R0` and `R1` after the call.

(a) Which register holds the return address after `BL foo`?

(b) Are `R0` and `R1` caller-saved or callee-saved in the lecture convention?

(c) What must the caller do before the call if it still needs those values afterwards?

B3. Stack-pointer updates (8 marks)
Assume the stack is full descending and initially `SP = 0x1000`.

(a) What is `SP` after `PUSH {R4, R5, LR}`?

(b) What is `SP` after then executing `SUB SP, SP, #8` for two 32-bit locals?

(c) If the function later executes `ADD SP, SP, #8` and `POP {R4, R5, LR}`, what is the final `SP`?

B4. Prologue and epilogue reasoning (8 marks)
A function uses registers `R4` and `R7`, returns in `R0`, and makes a nested function call.

(a) Explain why `LR` must be preserved.

(b) Which of `R4` and `R7` must be saved if the function modifies them?

(c) Write a possible prologue and epilogue using `PUSH`/`POP`.

B5. Stack frames in recursion (8 marks)
A recursive function saves `R0` and `LR` using `PUSH {R0, LR}` on each call. Assume the initial `SP = 0x8000` and the function is called with `n = 3`, leading to active calls for `n=3`, `n=2`, and `n=1`.

(a) How many active stack frames exist at the deepest point?

(b) How many bytes have been consumed by those saves alone?

(c) What is `SP` at the deepest point?

B6. Caller-save vs callee-save (10 marks)
Suppose function `main` stores a useful temporary in `R2`, calls `f`, and later still needs the old value of `R2`. Function `f` modifies `R4`, `R5`, and `R2`, and then calls `g`.

(a) Which function is responsible for preserving the old value of `R2` across the call to `f`?

(b) Which function is responsible for preserving `R4` and `R5` if `f` changes them?

(c) Why is `f` also responsible for preserving `LR` before calling `g`?

(d) Briefly explain why this division of responsibility is more efficient than "save every register every time".

Part C: Past Exam Questions (Source-Tagged) (20 marks)

C1. Functions with stack support (Source: 2023_Final_Exam_SuppDA.pdf, Part A, adapted) (6 marks)
A teaching ISA is extended with `sp`, `lr`, `push`, `pop`, `bl`, and `bx lr`. Explain why each of the following is useful for implementing functions:

(a) An explicit link register

(b) Stack push/pop support

(c) A branch-and-link style call instruction

C2. Calling convention design (Source: 2023_Final_Exam_SuppDA.pdf, Part A, adapted) (7 marks)
Suppose a simple ISA has a function `mul` that may clobber registers. Propose a small calling convention for four general-purpose registers and state clearly which are caller-saved and which are callee-saved. Briefly justify your design.

C3. Returning without a link register (Source: 2023_Final_Exam.pdf, Q1A, adapted) (7 marks)
Explain how functions can still be implemented in an ISA that has a stack pointer but no dedicated link register. What must be saved on the stack at call time, and how is the return performed?